

Value of information

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	The umbrella problem	2
4	A sequential problem: test then act	3
5	Repairing a diagram that forgets	4
6	Grazing: the value of a vegetation survey	6
7	Monotonicity	7
8	Summary	7
9	References	8

1 Overview

Because information sets are explicit objects of the schema, the value of observing a variable before a decision is a difference of two optimisations (SPEC section 35):

$$EVI(X \rightarrow D) = \max_{\sigma} EU(ID + info(X, D)) - \max_{\sigma} EU(ID).$$

Adding an information arc enlarges the class of admissible policies, so the value is never negative (SPEC section 55.7; the Lean theorem `exists_strategy_enlarged_eq` shows that every strategy of the smaller diagram has an equivalent in the larger one). The idea is Howard (1966)'s; Raiffa (1968) is the classical treatment of the test-then-act problem used below, and Runge et al. (2011) is the ecological case for computing it before commissioning monitoring.

2 Setup

```
using InfluenceDiagrams
using Random
```

3 The umbrella problem

Start from the umbrella problem without any information: the decision maker chooses once and for all. The best they can do is leave the umbrella at home, worth 70.

```
m = umbrella_model()
noinfo = without_information(m, :Umbrella, :Forecast)
information_names(syntax(noinfo), :Umbrella), optimize(noinfo).expected_utility
```

```
(Symbol[], 70.0)
```

`with_information` adds an arc; the validity of the enlarged diagram is checked (the variable must be available before the decision) and any policy whose signature changed is dropped from the strategy. Observing the forecast raises the optimum to 77, so the forecast is worth 7:

```
withf = with_information(noinfo, :Umbrella, :Forecast)
optimize(withf).expected_utility
```

```
77.000000000000001
```

```
expected_value_of_information(noinfo, :Forecast, :Umbrella)
```

```
7.0000000000000014
```

Observing the weather itself is perfect information: the optimum becomes 91 and the value 21. `expected_value_of_perfect_information` adds every admissible chance variable (not already observed and not downstream of the action).

```
expected_value_of_information(noinfo, :Weather, :Umbrella)
```

```
21.0
```

```
variable_name.(Ref(syntax(noinfo)), admissible_information(noinfo, :Umbrella))
```

```
2-element Vector{Symbol}:
 :Weather
 :Forecast
```

```
expected_value_of_perfect_information(noinfo, :Umbrella)
```

21.0

Relative to the diagram that already has the forecast, the weather is worth the remaining 14; a variable already observed is worth nothing; a variable downstream of the action cannot be observed.

```
expected_value_of_perfect_information(m, :Umbrella),  
expected_value_of_information(m, :Forecast, :Umbrella)
```

(13.999999999999986, 0.0)

```
try  
  expected_value_of_information(two_stage_model(), :Result, :Test)  
catch e  
  e  
end
```

InvalidInformationSetError(:Test, :Result, :downstream)

Both backends give the same values; the exhaustive one is the oracle.

```
expected_value_of_information(noinfo, :Forecast, :Umbrella; backend =  
ExhaustivePolicySearch())
```

7.0

4 A sequential problem: test then act

The umbrella problem has one decision. The interesting case is a sequential one, where information acquired before an early decision has to be carried forward. `two_stage_model` is a small oil wildcatter ([Raiffa 1968](#)): decide whether to Test, observe a Result, then decide whether to Drill. Testing costs 10, a wet site pays 100 and a dry one costs 60.

```
t = two_stage_model()  
sol = optimize(t)  
(meu = sol.expected_utility, test = policy_table(sol.strategy[:Test]),  
 drill = policy_table(sol.strategy[:Drill]))
```

```
(meu = 21.0, test = fill(:test), drill = [:drill :dont :drill; :drill :drill  
:drill])
```

The optimum is 21: test, then drill exactly on a positive result. What would it be worth to know the oil state before deciding whether to test at all?

```
expected_value_of_information(t, :Oil, :Test)
```

29.0

Twenty-nine, which takes the optimum to 50: a decision maker who already knows the state skips the test and drills exactly when the site is wet. The important part is how `with_information` builds the enlarged diagram. Under perfect recall (no-forgetting), a variable observed before an early decision is still known at every later one, so the arc is propagated to the whole of the rest of the decision order:

```
enlarged = with_information(t, :Test, :Oil)
(test = information_names(syntax(enlarged), :Test),
 drill = information_names(syntax(enlarged), :Drill),
 no_forgetting = is_no_forgetting(enlarged))
```

```
(test = [:Oil], drill = [:Test, :Result, :Oil], no_forgetting = true)
```

Without that propagation the enlarged diagram would forget: `Test` would know the oil state and `Drill` would not. `no_forgetting = false` asks for exactly that, which is a legitimate limited-memory diagram ([Lauritzen and Nilsson 2001](#)) but a different, and lower-valued, problem: the only way to get the oil state to the drilling decision is to signal it through the test.

```
single = with_information(t, :Test, :Oil; no_forgetting = false)
(drill = information_names(syntax(single), :Drill),
 no_forgetting = is_no_forgetting(single),
 optimum = optimize(single, ExhaustivePolicySearch()).expected_utility)
```

```
(drill = [:Test, :Result], no_forgetting = false, optimum = 45.0)
```

Both backends agree on the propagated diagram, and the expected value of perfect information for the first decision is the same 29 (the oil state is the only admissible chance variable there: `Result` is downstream of `Test`).

```
(dve = expected_value_of_information(t, :Oil, :Test),
 exhaustive = expected_value_of_information(t, :Oil, :Test; backend =
ExhaustivePolicySearch()),
 evpi = expected_value_of_perfect_information(t, :Test),
 admissible = variable_name.(Ref(syntax(t)), admissible_information(t, :Test)))
```

```
(dve = 29.0, exhaustive = 29.0, evpi = 29.0, admissible = [:Oil])
```

5 Repairing a diagram that forgets

Decision variable elimination is exact only on diagrams with no-forgetting: when a decision is maximised out, the value potential must depend on nothing but that decision's information set. This is *not* a validity condition. A diagram that forgets is a well-formed limited-memory

influence diagram, and ExhaustivePolicySearch solves it exactly; validate therefore accepts it. Take the two-stage diagram and delete the memory of the test decision:

```
forgetful = without_information(t, :Drill, :Test)
(valid = validate(syntax(forgetful)) == nothing,
 missing_arcs = no_forgetting_arcs(forgetful))
```

```
(valid = true, missing_arcs = [:Drill => :Test])
```

no_forgetting_arcs lists what is missing as decision $\Rightarrow$ variable pairs. The default backend refuses the diagram and says so, naming the variables the decision maker would have to remember:

```
try
    optimize(forgetful)
catch e
    println(sprint(showerror, e))
end
```

IrregularDiagramError: when decision :Drill is maximised, the utility potential still depends on Test, which the decision maker does not observe; decision variable elimination is exact only for diagrams with no-forgetting (perfect recall). Add the missing information arcs with with_no_forgetting, or solve this limited-memory diagram with ExhaustivePolicySearch

with_no_forgetting adds exactly the listed arcs (and drops the policies whose signature changed), after which the two backends agree again:

```
repaired = with_no_forgetting(forgetful)
(drill = information_names(syntax(repaired), :Drill),
 dve = optimize(repaired).expected_utility,
 exhaustive = optimize(repaired, ExhaustivePolicySearch()).expected_utility,
 limited_memory = optimize(forgetful, ExhaustivePolicySearch()).expected_utility)
```

```
(drill = [:Result, :Test], dve = 21.0, exhaustive = 21.0, limited_memory = 21.0)
```

The same repair is what makes two decisions with no arcs between them solvable. decision_order linearises a partial order by part id, so such decisions are ordered but neither knows the other's action; that is a forgetting diagram, and with_no_forgetting wires the earlier action into the later information set. Shachter (1986) calls a diagram with a directed path through all the decisions *regular*; no-forgetting is the stronger property, and it is the one the algorithm needs.

```
unordered = influence_diagram(:A => [:a1, :a2], :B => [:b1, :b2], :X => [:x1, :x2];
    mechanisms = [:X => (:A, :B)],
    decisions = [:A => Symbol[], :B => Symbol[]],
    utilities = [:U => :X])
tab = zeros(2, 2, 2)
tab[1, 1, :] = [0.9, 0.1]; tab[1, 2, :] = [0.2, 0.8]
```

```

tab[2, 1, :] = [0.5, 0.5]; tab[2, 2, :] = [0.1, 0.9]
um = bind_utility(bind_cpt(InfluenceDiagramModel(unordered), :X ⇒ tab), :U ⇒ [0.0,
10.0])
(missing_arcs = no_forgetting_arcs(um),
 exhaustive = optimize(um, ExhaustivePolicySearch()).expected_utility,
 repaired = optimize(with_no_forgetting(um)).expected_utility)

```

```

(missing_arcs = [:B => :A], exhaustive = 9.0, repaired = 9.0)

```

6 Grazing: the value of a vegetation survey

The SPEC section 46 diagram (next vignette) informs the grazing decision with a climate forecast and a survey of the current vegetation. Analyses 3 and 4 ask what happens when the survey is dropped, and what it is worth. With the numbers of the grazing_reference_id fixture the management cost dominates the conservation benefit (at most 100), the optimal policy is maintain whatever the information, and no information has any value:

```

g = reference_grazing_model()
without = without_information(g, :GrazingManagement, :CurrentVegetation)
(with_survey = optimize(g).expected_utility, without_survey =
optimize(without).expected_utility,
 policy = unique(policy_table(optimize(g).strategy[:GrazingManagement])))

```

```

(with_survey = 36.524146862500004, without_survey = 36.524146862500004, policy =
[:maintain])

```

```

expected_value_of_information(without, :CurrentVegetation, :GrazingManagement)

```

```

0.0

```

Information is only worth something when it can change the decision. Valuing high biodiversity at 1000 instead of 100 (bind_utility replaces the utility of the ConservationBenefit node) makes the policy react to both signals, and the survey, the forecast and the unobservable variables all acquire a value:

```

h = bind_utility(g, :ConservationBenefit ⇒ [0.0, 1000.0])
policy_table(optimize(h).strategy[:GrazingManagement])

```

```

3×3 Matrix{Symbol}:
:reduce  :exclude  :exclude
:exclude  :exclude  :reduce
:exclude  :exclude  :reduce

```

```

hw = without_information(h, :GrazingManagement, :CurrentVegetation)
(survey = expected_value_of_information(hw, :CurrentVegetation, :GrazingManagement),
 forecast = expected_value_of_information(without_information(h, :GrazingManagement,
:ClimateForecast),
                                     :ClimateForecast, :GrazingManagement),
 climate = expected_value_of_information(h, :Climate, :GrazingManagement),
 soil = expected_value_of_information(h, :SoilMoisture, :GrazingManagement),
 perfect = expected_value_of_perfect_information(h, :GrazingManagement))

```

```

(survey = 0.424781849999988, forecast = 0.101836350000004188, climate =
0.18480577499991568, soil = 0.6800549000000016, perfect = 0.6800549000000016)

```

Perfect information about everything upstream of the decision bounds what any monitoring programme could be worth; here soil moisture alone captures all of it.

7 Monotonicity

The property that information never hurts is tested on random tiny diagrams in the test suite (`test/test_information.jl`); here is one draw of the same experiment.

```

rng = MersenneTwister(3)
values = Float64[]
for _ in 1:8
    id = influence_diagram(:A => [:a1, :a2, :a3], :B => [:b1, :b2], :D => [:d1,
:d2];
                                mechanisms = [:B => :A], decisions = [:D => Symbol[]],
                                utilities = [:U => (:A, :D)])
    mm = InfluenceDiagramModel(id)
    mm = bind_kernel(mm, :A => random_kernel(rng, FiniteSpace(), space(mm, :A)))
    mm = bind_kernel(mm, :B => random_kernel(rng, space(mm, :A), space(mm, :B)))
    mm = bind_utility(mm, :U => 10 .* rand(rng, 3, 2))
    push!(values, expected_value_of_information(mm, :B, :D))
end
all(values .>= -1e-12), round.(values; digits = 3)

```

```

(true, [0.0, 0.0, -0.0, 0.312, 0.0, 0.288, 0.0, -0.0])

```

8 Summary

- The value of information is a difference of two optimisations over two ACSets, because information sets are objects of the schema rather than a solver option.
- `with_information` propagates the new arc to every later decision, which is what `info(X, D)` means under perfect recall; `no_forgetting = false` keeps the single-arc, limited-memory reading, which is a different problem with a different value (50 against 45 on the two-stage diagram).

- Decision variable elimination needs no-forgetting; validate does not, because a diagram that forgets is a legitimate limited-memory diagram. `no_forgetting_arcs` reports what is missing, `with_no_forgetting` supplies it, and the solver's `IrregularDiagramError` points at both.
- Information is worth nothing when it cannot change the decision: the SPEC section 46 reference numbers are degenerate in exactly that way, and the values only appear once the conservation benefit is large enough to make the policy react.
- Information never hurts: every value above is non-negative, as the Lean theorem `exists_strategy_enlarged_eq` and the random-diagram property tests require.

9 References

Howard (1966) introduced the value of information as a decision-analytic quantity, and Raiffa (1968) is the classical text in which the test-then-act problem used here appears. Shachter (1986) defines influence diagrams, their evaluation, and the regularity and no-forgetting conditions distinguished above; Lauritzen and Nilsson (2001) study the limited-memory diagrams that arise when no-forgetting is dropped, which is what `ExhaustivePolicySearch` still solves here. Runge et al. (2011) make the case for computing the value of information before investing in monitoring, and is the source of the framing of the grazing example.

Howard, Ronald A. 1966. "Information Value Theory." *IEEE Transactions on Systems Science and Cybernetics* 2 (1): 22–26. <https://doi.org/10.1109/TSSC.1966.300074>.

Lauritzen, Steffen L., and Dennis Nilsson. 2001. "Representing and Solving Decision Problems with Limited Information." *Management Science* 47 (9): 1235–51. <https://doi.org/10.1287/mnsc.47.9.1235.9779>.

Raiffa, Howard. 1968. *Decision Analysis: Introductory Lectures on Choices Under Uncertainty*. Addison-Wesley.

Runge, Michael C., Sarah J. Converse, and James E. Lyons. 2011. "Which Uncertainty? Using Expert Elicitation and Expected Value of Information to Design an Adaptive Program." *Biological Conservation* 144 (4): 1214–23. <https://doi.org/10.1016/j.biocon.2010.12.020>.

Shachter, Ross D. 1986. "Evaluating Influence Diagrams." *Operations Research* 34 (6): 871–82. <https://doi.org/10.1287/opre.34.6.871>.